13. Keylime 설정 관리 및 배포 방식 분석

13.1 설정 파일 파싱 및 환경 변수 처리 (`keylime/config.py`)

13.1.1 설정 시스템 아키텍처

Keylime은 계층적 설정 시스템을 사용하여 유연하고 확장 가능한 구성 관리를 제공합니다. 설정 우선순위는 다음과 같습니다:

1. **환경 변수** (최고 우선순위)
2. **사용자 설정 파일** (`/etc/keylime/`)
3. **시스템 기본 설정** (`/usr/etc/keylime/`)
4. **설정 스니펫** (`.conf.d` 디렉토리)

13.1.2 핵심 설정 함수 분석

환경 변수 처리 함수

```
def  environ_bool(env_name: str, default: bool) -> bool:
"""환경 변수를 불린 값으로 변환"""
val = os.getenv(env_name, "default").lower()
if val in ["on", "true", "1"]:
return True
if val in ["off", "false", "0"]:
return False
if val == "default":
```

```
return default
raise ValueError(f"Environment variable {env_name} set to invalid value {val}")
```

데이터 변환 함수

```
def convert(data: Any) -> Any:
"""바이트, 딕셔너리, 튜플, 리스트 등의 데이터 타입을 재귀적으로 변환"""
if isinstance(data, bytes):
return data.decode()
if isinstance(data, dict):
return dict(iter(map(convert, data.items())))
# ... 추가 타입 처리
```

13.1.3 설정 파일 경로 구조

기본 설정 파일 경로

```
CONFIG_FILES = {
"verifier": ["/etc/keylime/verifier.conf", "/usr/etc/keylime/verifier.conf"],
"tenant": ["/etc/keylime/tenant.conf", "/usr/etc/keylime/tenant.conf"],
"registrar": ["/etc/keylime/registrar.conf", "/usr/etc/keylime/registrar.conf"],
"ca": ["/etc/keylime/ca.conf", "/usr/etc/keylime/ca.conf"],
"logging": ["/etc/keylime/logging.conf", "/usr/etc/keylime/logging.conf"],
```

```
}
```

설정 스니펫 디렉토리

```
CONFIG_SNIPPETS_DIRS = {
"verifier": ["/usr/etc/keylime/verifier.conf.d", "/etc/keylime/verifier.conf.d"]
"tenant": ["/usr/etc/keylime/tenant.conf.d", "/etc/keylime/tenant.conf.d"],
# ... 각 컴포넌트별 스니펫 디렉토리
}
```

13.1.4 환경 변수 기반 설정 오버라이드

Keylime은 각 설정 옵션에 대해 환경 변수를 통한 오버라이드를 지원합니다:

```
def _get_env(component: str, option: str, section: Optional[str]) -> Optional[s
"""환경 변수를 통한 설정 오버라이드 처리"""
opt_section = f"_{section.upper()}" if section else ""
env_name = f"KEYLIME_{component.upper()}{opt_section}_{option.upper()}"
env_value = os.environ.get(env_name, None)
if env_value is not None:
log_msg = f'option "{option}" on section {section} for component {component}.con
base_logger.info(log_msg.replace("on section None ", ""))
return env_value
```

13.1.5 설정 버전 관리 및 업그레이드

```
def  check_version(component: str, logger: Optional[logging.Logger] = None) -> b
"""설정 파일 버전 확인 및 업그레이드 가능 여부 검사"""
config_version = get(component, "version", fallback="1.0")
cur_version = str_to_version(config_version)
# 최신 버전과 비교하여 업그레이드 필요성 판단
if cur_version < latest:
if cur_version[0] < latest[0]:
# 메이저 업데이트 경고
logger.warning("A major configuration upgrade is available")
return True
```

13.2 설정 파일 구조 분석 (`/etc/keylime/*.conf`)

13.2.1 Verifier 설정 구조 (`verifier.conf`)

기본 서버 설정

```
[verifier]
version = 2.5
uuid = d432fbb3-d2f1-4a97-9ef7-75bd81c00000
ip = "127.0.0.1"
port = 8881
```

```
registrar_ip = 127.0.0.1

registrar_port = 8891
```

TLS 및 보안 설정

```
# mTLS 통신 활성화

enable_agent_mtls = True


# TLS 디렉토리 및 인증서 설정

tls_dir = generate

server_key = default

server_cert = default

trusted_client_ca = default
```

데이터베이스 설정

```
# SQLite 기본 설정

database_url = sqlite:////var/lib/keylime/cv_data.sqlite


# MySQL 예시 설정

# database_url = mysql+pymysql://keylime:keylime@keylime_db:3306/verifier?charse


database_pool_sz_ovfl = 5,10
```

```
auto_migrate_db = True
```

검증 정책 설정

```
# 부팅 증명 정책
measured_boot_policy_name = accept-all
measured_boot_evaluate = once

# IMA 오류 처리
ignore_tomtou_errors = True

# 허용 목록 서명 요구
require_allow_list_signatures = False
```

13.2.2 Registrar 설정 구조 (`registrar.conf`)

서버 바인딩 설정

```
[registrar]
version = 2.5
ip = "127.0.0.1"
port = 8890
tls_port = 8891
```

TPM 신원 확인 정책

```
# TPM 기반 신원 확인 방식
tpm_identity = default # ek_cert_or_iak_idevid, ek_cert, iak_idevid


# 잘못된 인증서 처리 방식
malformed_cert_action = warn # reject, ignore
```

13.2.3 CA 설정 구조 (ca.conf)

인증서 발급 설정

```
[ca]
password = default
cert_country = US
cert_ca_name = Keylime Certificate Authority
cert_state = MA
cert_locality = Lexington
cert_organization = MITLL
cert_org_unit = 53
```

인증서 유효기간 설정

```
cert_ca_lifetime = 3653 # CA 인증서 유효기간 (일)
cert_lifetime = 365 # 일반 인증서 유효기간 (일)
```

```
cert_bits = 2048 # 키 길이 (비트)
```

13.3 SystemD 서비스 파일 분석 (`services/`)

13.3.1 Verifier 서비스 설정

서비스 파일 구조 (`keylime_verifier.service`)

```
[Unit]
Description=The Keylime verifier
After=network.target
Before=keylime_registrar.service
StartLimitInterval=10s
StartLimitBurst=5

[Service]
Group=keylime
User=keylime
ExecStart=/usr/bin/keylime_verifier
Restart=on-failure
TimeoutSec=60s
RestartSec=120s

[Install]
```

```
WantedBy=default.target
```

서비스 의존성 관리

- **After=network.target**: 네트워크 서비스 시작 후 실행
- **Before=keylime_registrar.service**: Registrar보다 먼저 시작
- **StartLimitBurst=5**: 10초 내 최대 5회 재시작 시도

13.3.2 Registrar 서비스 설정

서비스 파일 구조 (`keylime_registrar.service`)

```
[Unit]
Description=The Keylime registrar service
After=network.target keylime_verifier.service
StartLimitInterval=10s
StartLimitBurst=5

[Service]
Group=keylime
User=keylime
ExecStart=/usr/bin/keylime_registrar
TimeoutSec=60s
Restart=on-failure
RestartSec=120s
```

```
[Install]
WantedBy=default.target
```

13.3.3 서비스 설치 스크립트 (`installer.sh`)

사용자 및 디렉토리 생성

```
# keylime 시스템 사용자 생성
if ! getent passwd keylime >/dev/null; then
adduser --system --shell /bin/false \
--home /var/lib/keylime --no-create-home \
keylime
fi

# 필요한 디렉토리 생성
mkdir -p /var/lib/keylime
mkdir -p /var/log/keylime
mkdir -p /var/run/keylime
```

권한 설정

```
# 파일 소유권 설정
chown keylime:keylime -R /etc/keylime
chown keylime:keylime -R /var/lib/keylime
```

```
chown keylime:keylime -R /var/log/keylime
chown keylime:keylime -R /var/run/keylime

# 보안 권한 설정
chmod 700 /var/run/keylime
chmod 664 /etc/systemd/system/keylime_*.service
```

서비스 활성화

```
# 부팅 시 자동 시작 설정
systemctl enable keylime_registrar.service
systemctl enable keylime_verifier.service
```

13.4 Docker 컨테이너 기반 배포 옵션

13.4.1 Docker 이미지 구조

베이스 이미지 (`docker/release/base/Dockerfile.in`)

```
FROM quay.io/fedora/fedora@sha256:cde80fcd3e22f667679e1eb6454c30c96a3b4043ee3069
LABEL version="_version_" description="Keylime Base - Only used as an base image

RUN dnf -y install \
awk \
dnf-plugins-core \
```

```
efivar-devel \
efivar-libs \
git \
nc \
python3-PyMySQL \
python3-jinja2 \
python3-psycopg2 \
tpm2-tools
```

Verifier 컨테이너 (`docker/release/verifier/Dockerfile.in`)

```
FROM _source_keylime_base_digest_ AS keylime_verifier
LABEL version="_version_" description="Keylime Verifier - Bootstrapping and Main

RUN --mount=target=/keylime,type=bind,source=.,rw \
cd /keylime && ./installer.sh -o

EXPOSE 8880
EXPOSE 8881

ENTRYPOINT ["keylime_verifier"]
```

Registrar 컨테이너 (`docker/release/registrar/Dockerfile.in`)

```
FROM _source_keylime_base_digest_ AS keylime_registrar

LABEL version="_version_" description="Keylime Registrar - Bootstrapping and Mai



EXPOSE 8890

EXPOSE 8891



ENTRYPOINT ["keylime_registrar"]
```

13.4.2 Docker 빌드 스크립트 분석

로컬 빌드 스크립트 (`build_locally.sh`)

```
# 컨테이너 엔진 자동 감지

if [ -z "$CONTAINER_ENGINE" ];  then

if  command  -v  docker &> /dev/null;  then

CONTAINER_ENGINE="docker"

elif  command  -v  podman &> /dev/null;  then

CONTAINER_ENGINE="podman"

else

echo  "ERROR: No container engine specified"

exit  1

fi

fi
```

```
# Fedora 베이스 이미지 다이제스트 확인
FEDORA_IMAGE="${REGISTRY}/fedora/fedora"
QUERY_RESULT="$(skopeo inspect docker://${FEDORA_IMAGE}:latest)"
FEDORA_DIGEST="$(jq '.Digest' <<<"$QUERY_RESULT")"
```

이미지 생성 프로세스

```
# 각 컴포넌트별 이미지 빌드
for part in base registrar verifier tenant; do
${CONTAINER_ENGINE} buildx build -t keylime_${part}:${VERSION} \
-f "${part}/Dockerfile" \
--security-opt label=disable \
--progress plain ${@:3} "$KEYLIME_DIR"
done
```

13.4.3 컨테이너 네트워킹 및 포트 설정

기본 포트 매핑

- **Verifier**: 8880 (HTTP), 8881 (HTTPS)
- **Registrar**: 8890 (HTTP), 8891 (HTTPS)
- **Agent**: 9002 (기본 포트)

Docker Compose 예시 설정

```yaml
version: '3.8'

services:

keylime-verifier:

image: keylime_verifier:latest

ports:

- "8881:8881"

volumes:

- keylime-data:/var/lib/keylime

environment:

- KEYLIME_VERIFIER_DATABASE_URL=sqlite:////var/lib/keylime/cv_data.sqlite

keylime-registrar:

image: keylime_registrar:latest

ports:

- "8891:8891"

depends_on:

- keylime-verifier
```

13.5 개발 및 프로덕션 환경 설정 차이점

13.5.1 개발 환경 설정

테스트 모드 활성화

```
# 환경 변수를 통한 테스트 모드 설정
```

```
TEST_MODE = environ_bool("KEYLIME_TEST", False)

if TEST_MODE:

print("WARNING: running keylime in testing mode.")

DISABLE_EK_CERT_CHECK_EMULATOR = True

WORK_DIR = os.getenv("KEYLIME_DIR", os.getcwd())
```

개발 환경 특징

- **EK 인증서 검증 비활성화**: TPM 에뮬레이터 사용 시
- **보안 마운트 생성 비활성화**: 개발 편의성 향상
- **작업 디렉토리 변경**: 현재 디렉토리 사용

디버그 설정

```
# 디버그 모드 설정

INSECURE_DEBUG = False # 프로덕션에서는 반드시 False

DEBUG_DB = environ_bool("DEBUG_DB", False) # 데이터베이스 디버깅
```

13.5.2 프로덕션 환경 설정

보안 강화 설정

```
# mTLS 필수 활성화

enable_agent_mtls = True


# 허용 목록 서명 요구
```

```
require_allow_list_signatures = True


# EK 인증서 검증 활성화
DISABLE_EK_CERT_CHECK_EMULATOR = False
```

데이터베이스 설정

```
# 프로덕션용 MySQL/PostgreSQL 사용
database_url = mysql+pymysql://keylime:secure_password@db_host:3306/keylime_veri


# 연결 풀 최적화
database_pool_sz_ovfl = 20,50
```

로깅 및 모니터링

```
# 구조화된 로깅 설정
[logging]
level = INFO
format = json
handlers = file,syslog


# 성능 모니터링
quote_interval = 2.0
```

```
request_timeout = 70.0

max_retries = 5
```

13.5.3 환경별 설정 관리 전략

환경 변수 기반 설정

```
# 개발 환경
export KEYLIME_TEST=true
export KEYLIME_VERIFIER_DATABASE_URL="sqlite:///./dev_data.sqlite"
export KEYLIME_VERIFIER_ENABLE_AGENT_MTLS=false

# 프로덕션 환경
export KEYLIME_VERIFIER_DATABASE_URL="mysql+pymysql://keylime:${DB_PASSWORD}@${
export KEYLIME_VERIFIER_ENABLE_AGENT_MTLS=true
export KEYLIME_VERIFIER_REQUIRE_ALLOW_LIST_SIGNATURES=true
```

설정 스니펫을 통한 환경별 오버라이드

```
# 개발 환경 스니펫 (/etc/keylime/verifier.conf.d/01-dev.conf)
[verifier]
quote_interval = 10.0
max_retries = 2
```

```
# 프로덕션 환경 스니펫 (/etc/keylime/verifier.conf.d/01-prod.conf)

[verifier]

quote_interval = 2.0

max_retries = 10

num_workers = 4
```

13.5.4 컨테이너 환경별 설정

개발용 Docker Compose

```yaml
version: '3.8'

services:

keylime-dev:

build: .

environment:

- KEYLIME_TEST=true

- KEYLIME_VERIFIER_DATABASE_URL=sqlite:////tmp/dev_data.sqlite

volumes:

- .:/keylime:rw

ports:

- "8881:8881"
```

프로덕션용 Kubernetes 배포

```yaml
apiVersion: apps/v1
```

```yaml
kind: Deployment

metadata:

name: keylime-verifier

spec:

replicas: 3

selector:

matchLabels:

app: keylime-verifier

template:

spec:

containers:

- name: verifier

image: keylime_verifier:2.5.0

env:

- name: KEYLIME_VERIFIER_DATABASE_URL

valueFrom:

secretKeyRef:

name: keylime-db-secret

key: database-url

resources:

requests:

memory: "512Mi"

cpu: "500m"
```

```
limits:

memory: "1Gi"

cpu: "1000m"
```

이 분석을 통해 Keylime의 설정 관리 시스템이 환경 변수, 설정 파일, 스니펫을 통한 계층적 구성을 지원하며, 개발부터 프로덕션까지 다양한 배포 시나리오를 지원함을 확인할 수 있습니다. SystemD 서비스와 Docker 컨테이너 기반 배포 모두를 지원하여 유연한 인프라 구성이 가능합니다.